

CSS object-position

Master image positioning and focal point control with object-position

What is CSS object-position?

The `object-position` property specifies the alignment of a replaced element (like an image or video) within its container. It works in conjunction with `object-fit` to control exactly where content appears when it's being scaled, cropped, or fitted into a container.

Think of `object-position` as the CSS equivalent of the `background-position` property, but specifically designed for images, videos, and other replaced elements.

Browser Support: All modern browsers (Chrome, Firefox, Safari, Edge) support `object-position`. IE 11 does not support it. Always provide fallbacks for older browsers.

Core Concepts & Relationship with object-fit

How object-position Works

`object-position` defines the placement point of replaced content within its container. It's only visible when:

- Using `object-fit: cover` (content is clipped, positioning shows what's visible)
 - Using `object-fit: contain` (content is smaller than container, positioning shows alignment)
 - Using `object-fit: none` (original size with clipping)
- `object-position` has NO effect with `object-fit: fill` (default) because the content stretches to fill the entire container.

object-position vs background-position

Aspect object-position background-position Used with Replaced elements (, <video>) Background images Semantic Yes - content is part of page No - decorative only Accessibility Content is in DOM Not accessible Use Case Images, videos, content Decorative backgrounds

object-position Syntax & Values

Basic Syntax

```
Syntax: object-position: <x-position> <y-position>;

Default Value: 50% 50% (center)

Value Types:
• Keywords: left, right, center, top, bottom
• Percentages: 0% to 100%
• Pixels: any absolute length value
```

Keyword Values

Keywords provide readable, intuitive positioning:

top left

top center

top right

center left

center

center right

bottom left

bottom center

bottom right

Percentage Values

Percentages are relative to the container's dimensions. 0% 0% is top-left, 100% 100% is bottom-right.

```
/* X-axis (horizontal) from left (0%) to right (100%) */
/* Y-axis (vertical) from top (0%) to bottom (100%) */

object-position: 0% 0%; /* Top-left corner */
object-position: 50% 50%; /* Center (default) */
object-position: 100% 100%; /* Bottom-right corner */
object-position: 25% 75%; /* Custom position */
object-position: 0% 50%; /* Left center */
object-position: 100% 50%; /* Right center */
```

Pixel Values

Absolute pixel measurements from the top-left corner:

```
object-position: 10px 20px; /* 10px from left, 20px from top */
object-position: 50px 0px; /* 50px from left, at top */
object-position: 0px 30px; /* At left, 30px from top */
```

Mixed Values

You can combine different value types:

```
object-position: 50% 20px; /* Center horizontally, 20px from top */
object-position: 10px center; /* 10px from left, vertically centered */
object-position: right 10px; /* 10px from right, vertically centered */
```

Interactive object-position Demos

All containers use 200px × 160px with `object-fit: cover`. The gradient shows how positioning changes the visible area.

top left

0% 0%

top center

50% 0%

top right

100% 0%

center left

0% 50%

center (default)

50% 50%

center right

100% 50%

bottom left

0% 100%

bottom center

50% 100%

bottom right

100% 100%

Fine-Tuned Positioning Examples

Custom percentage values for precise focal point control

30% 40%

Custom position

75% 25%

Custom position

20% 80%

Custom position

Practical Examples & Use Cases

1. Hero Image with Focal Point on Face

Position focal point at top to show person's face prominently:

```
img.hero {
  width: 100%;
  height: 500px;
  object-fit: cover;
  object-position: center top; /* Focus on top (face) */
}
```

2. Gallery Thumbnail - Face Priority

Always show the face in portrait photos:

```
.gallery-thumb {
  width: 200px;
  height: 200px;
  object-fit: cover;
  object-position: center 35%; /* Show upper portion (face) */
}
```

3. Product Image - Show Center Details

For product photos, keep center detail visible:

```
.product-image {
  width: 300px;
  height: 300px;
  object-fit: cover;
  object-position: center; /* Default center positioning */
}
```

4. Landscape Photo - Preserve Horizon

Show more sky/foreground by positioning horizon line:

```
img.landscape {
  width: 100%;
  height: 300px;
  object-fit: cover;
  object-position: center 40%; /* Horizon at 40% from top */
}
```

5. Video Thumbnail - Show Important Content

Position where the action happens:

```
video.thumbnail {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  object-position: center 25%; /* Focus on upper area */
}
```

6. Profile Picture - Always Center

Ensure face is always in the middle:

```
.profile-pic {
  width: 100px;
  height: 100px;
  border-radius: 50%;
  object-fit: cover;
  object-position: center; /* Face centered */
}
```

7. Responsive Banner with Keyword Values

Use readable keyword positioning:

```
.banner {
  width: 100%;
  height: 300px;
  object-fit: cover;
  object-position: right center; /* Show right side of image */
}
```

8. Dynamic Positioning with CSS Variables

Allow positioning to be changed dynamically:

```
:root {
  --position-x: 50%;
  --position-y: 50%;
}

img {
  object-position: var(--position-x) var(--position-y);
}

/* JavaScript can update these variables */
img.focus-top {
  --position-x: 50%;
  --position-y: 25%;
}
```

Advanced Techniques

Animated Position Transitions

```
img {
  object-fit: cover;
  object-position: center;
  transition: object-position 0.5s ease;
}

img:hover {
  object-position: top; /* Pan to top on hover */
}
```

Keyframe Animation for Panning

```
@keyframes panImage {
  0% { object-position: left center; }
  50% { object-position: center center; }
  100% { object-position: right center; }
}

img.pan {
  animation: panImage 6s ease-in-out infinite;
}
```

Responsive Positioning

```
/* Mobile: show top (faces) */
@media (max-width: 768px) {
  img {
    object-position: center top;
  }
}

/* Desktop: show center */
@media (min-width: 769px) {
  img {
    object-position: center;
  }
}
```

Calculating Custom Positions

Use percentages based on visual hierarchy:

```
/* Rule of thirds positioning */
/* Place focal point at 1/3 from edges */
object-position: 33.33% 33.33%; /* Top-left third */
object-position: 66.67% 33.33%; /* Top-right third */
object-position: 33.33% 66.67%; /* Bottom-left third */
object-position: 66.67% 66.67%; /* Bottom-right third */
```

object-position with Different object-fit Values

With object-fit: cover

Most common combination. Content is clipped, positioning determines what's visible.

```
img {
  width: 300px;
  height: 200px;
  object-fit: cover; /* Clip content */
  object-position: center 30%; /* Show upper area */
}
```

With object-fit: contain

Entire image visible; positioning controls alignment of empty space.

```
img {
  width: 300px;
  height: 300px;
  object-fit: contain; /* No clipping */
  object-position: center; /* Center alignment */
}
```

With object-fit: none

Original image size maintained; positioning shows where clipping occurs.

```
img {
  width: 300px;
  height: 200px;
  object-fit: none; /* Keep original size */
  object-position: center 30%; /* Choose visible area */
}
```

With object-fit: fill

NO EFFECT: Content stretches to fill, positioning is ignored.

```
img {
  width: 300px;
  height: 200px;
  object-fit: fill; /* Stretches to fill */
  object-position: center; /* IGNORED */
}
```

With object-fit: scale-down

Image scaled only if needed; positioning matters when empty space exists.

```
img {
  width: 300px;
  height: 300px;
  object-fit: scale-down; /* Scale down if needed */
  object-position: top; /* Align to top if scaled */
}
```

Best Practices & Tips

- Always specify object-fit first:** Determine how content fits before positioning it.
- Match positioning to content:** Position focal points (faces, important details) in visible areas.
- Use keywords for clarity:** `center top` is more readable than `50% 0%` in most cases.
- Test on mobile:** Different screen sizes may crop differently. Ensure important content is visible.
- Combine with aspect-ratio:** Use `aspect-ratio` for consistent sizing with `object-fit` and `object-position`.
- Use CSS variables for flexibility:** Allow positioning to change via JavaScript or media queries.
- Consider landscape vs portrait:** Different positioning may be needed for different image orientations.
- Provide fallbacks:** IE 11 doesn't support `object-position`. Plan for graceful degradation.

Browser Compatibility

Modern Browsers: Chrome 32+, Firefox 36+, Safari 10+, Edge 79+ all support `object-position`.

Internet Explorer 11: Does NOT support `object-position`. Images will fall back to default positioning (centered).

Mobile Browsers: iOS Safari 10+, Android Chrome 32+ support `object-position`. Test on target devices.

Quick Reference - Position Values

Position	Keyword	Percentage	Use Case
Center (default)	<code>center</code>	<code>50% 50%</code>	Balanced composition
Top Center	<code>center top</code>	<code>50% 0%</code>	Show faces in portraits
Bottom Center	<code>center bottom</code>	<code>50% 100%</code>	Horizon in landscapes
Left Center	<code>left</code>	<code>0% 50%</code>	Show left side
Right Center	<code>right</code>	<code>100% 50%</code>	Show right side
Top Left	<code>left top</code>	<code>0% 0%</code>	Top-left corner
Top Right	<code>right top</code>	<code>100% 0%</code>	Top-right corner
Bottom Left	<code>left bottom</code>	<code>0% 100%</code>	Bottom-left corner
Bottom Right	<code>right bottom</code>	<code>100% 100%</code>	Bottom-right corner

Common Mistakes to Avoid

- Using without object-fit:** `object-position` has minimal effect without `object-fit: cover` or `object-fit: contain`.
- Using with object-fit: fill:** Positioning is completely ignored with the `fill` value.
- Forgetting the container size:** Container must have explicit width/height for positioning to work properly.
- Using only one value:** Always provide both X and Y positions (e.g., `center top`, not just `center`).
- Not considering responsiveness:** Positioning may need to change for different screen sizes.
- Complex percentage calculations:** Start with round numbers (0%, 25%, 50%, 75%, 100%) and adjust from there.
- Overusing precise pixels:** Percentages are more flexible and responsive than pixel values.
- Not testing focal points:** Verify that important content is visible on all screen sizes.

Complete Examples Summary

Profile Image Gallery

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
  gap: 15px;
}

.gallery img {
  width: 100%;
  aspect-ratio: 1 / 1;
  object-fit: cover;
  object-position: center top; /* Show faces */
}
```

Responsive Hero with Smart Positioning

```
.hero {
  width: 100%;
  height: 400px;
  object-fit: cover;
  object-position: center 35%;
}

@media (max-width: 768px) {
  .hero {
    height: 250px;
    object-position: center top; /* Show more details */
  }
}
```

Magazine Layout with Text Wrapping

```
.article-image {
  float: left;
  width: 250px;
  height: 300px;
  margin-right: 30px;
  object-fit: cover;
  object-position: center;
}
```